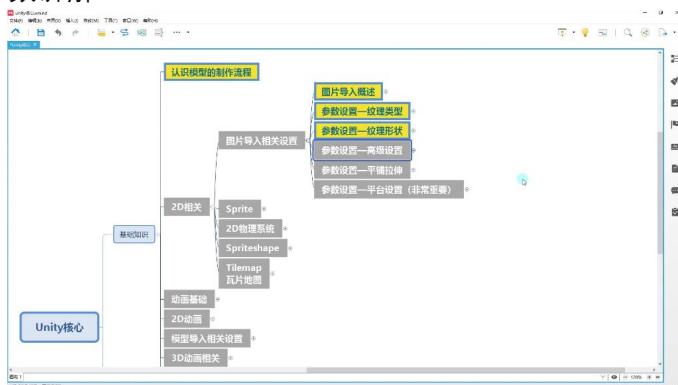


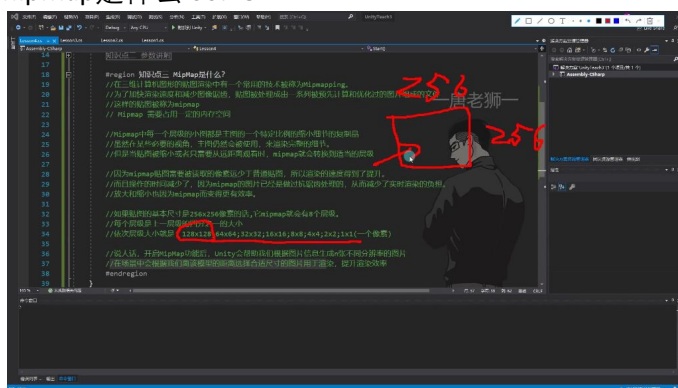
一、高级设置 00:04

1. 参数讲解 00:27

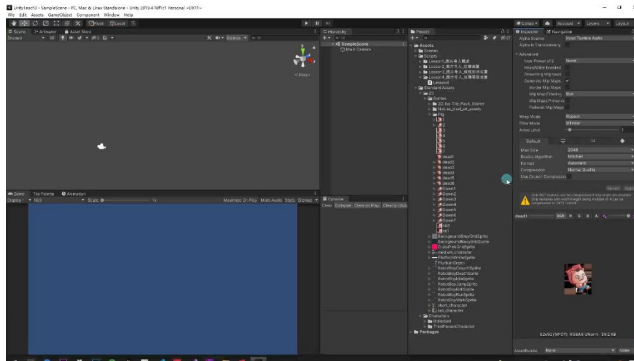


-
- **Non-Power of 2处理**: 当纹理尺寸非2的幂时, 提供四种处理方式:
 - None: 保持原尺寸
 - To nearest: 缩放到最接近的2的幂
 - To larger: 向上取整到更大的2的幂
 - To smaller: 向下取整到更小的2的幂
- **Read/Write Enabled**:
 - 启用后可通过Unity API获取纹理数据
 - 会增加内存占用, 非必要时应关闭

1) MipMap是什么 06:18

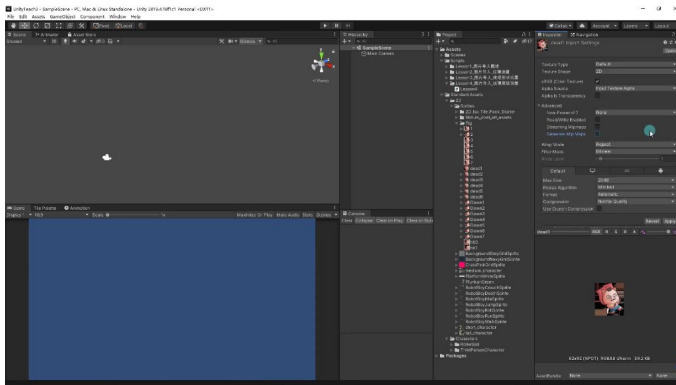


-
- **原理**: 预先生成多级缩小的纹理副本 (如256x256→128x128→64x64...→1x1)
- **作用**:
 - 提升远处物体的渲染性能
 - 减少锯齿现象
 - 典型的内存换性能方案
- **应用场景**:
 - 2D游戏通常不需要
 - 3D游戏根据性能需求选择启用
- **MipMap应用 12:32**



-
- **Streaming Mipmaps:**
 - 启用纹理串流技术
 - 减少内存占用约30-50%
- **Border Mip Maps:**
 - 防止颜色渗透到边缘
 - 建议保持启用
- **Filtering模式:**
 - Box: 平滑过渡
 - Kaiser: 锐化处理 (适用于模糊的远景)

2. 总结 14:44



-
- **核心原则:**
 - 2D项目禁用MipMap
 - 3D项目根据内存/性能平衡选择
- **关键参数:**
 - Generate Mip Maps: 总开关
 - Preserve Coverage: 保持Alpha通道完整性
 - Fadeout: 远景淡化 (通常禁用)
- **优化建议:**
 - PVRTC格式要求正方形纹理
 - 非必要不启用Read/Write
 - 移动端优先考虑Streaming选项

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
纹理尺寸处理规则	纹理尺寸需为2的幂次方(如128x128), 非2次幂时可选择: 保持原样/缩放至最近2次幂/最大/最小值处理	PVRTC格式要求正方形纹理 (长宽必须相同)	★★

Read/Write Enabled参数	控制是否允许通过代码读取纹理数据，启用会增加内存占用	仅在需要代码获取纹理数据时开启	★★
Mip Map核心机制	生成多级缩略图（如256x256→128x128→64x64...），根据距离自动切换层级提升渲染效率	本质是用内存换性能，2D项目通常禁用	★★★★
Mip Map生成参数	- Border Mip Maps防颜色渗透 - Filter Mode选择平滑/锐化算法 - Mipmap Fadeout可渐变为灰色	凯撒算法适用于需要锐化远处纹理的场景	★★★
纹理串流技术	通过Mipmap Streaming动态加载层级，减少内存占用	需配合Priority参数控制资源分配优先级	★★★
2D/3D项目选择建议	- 2D项目：建议禁用Mipmap - 3D项目：根据内存与性能平衡决策	关键权衡：内存占用 vs 渲染性能	★★★